

Bringing Flexible Data Placement to NVMeVirt: A Software-Defined FDP SSD for Storage Research

Sinu Lee
Dept. of Computer Science
and Engineering
Seoul National University

Bosung An
Dept. of Electrical and
Computer Engineering
Seoul National University

Jonghyeon Park
Dept. of Computer Science
and Engineering
Seoul National University

Abstract—Flexible Data Placement (FDP) is a recent NVMe feature that lets the host hint *where* data is placed on a flash device so that data of similar lifetime shares the same physically reclaimed region, reducing garbage-collection (GC) write amplification without abandoning the conventional block interface. Studying FDP, however, requires FDP-capable hardware, which remains scarce, and no open emulator exposes the feature to an unmodified host stack. We extend NVMeVirt, a software-defined virtual NVMe device implemented as a Linux kernel module, into a functional FDP SSD. It advertises the FDP capability, serves the FDP log pages and features, performs host-directed placement into per-Reclaim-Unit-Handle (RUH) reclaim units, and reports FDP events. The emulated device is driven end to end by the unmodified in-kernel NVMe driver and `nvme-cli`. Using a synthetic hot/cold workload that models two data lifetimes, we show that FDP placement reduces write amplification by up to 61% (a $2.5\times$ flash-endurance gain) relative to an *identical* device with placement disabled, and we characterize how the benefit depends on over-provisioning and workload skew. We then reproduce the effect on a RocksDB-like log-structured-merge workload, where mapping LSM levels to separate reclaim units cuts write amplification by 34% and reveals how many reclaim units a deployment actually needs. The extension and all evaluation artifacts are publicly available.

Index Terms—NVMe, Flexible Data Placement, flash storage, garbage collection, write amplification, SSD emulation.

I. INTRODUCTION

NAND flash cannot be overwritten in place: a page must be erased before reuse, and erases act on large blocks. Flash devices therefore write out-of-place and reclaim stale space with a garbage collector (GC) that must relocate any still-valid pages out of a victim block before erasing it. This relocation is pure overhead, since it spends flash program and erase cycles that do no useful host work. We capture it with the *write amplification factor* (WAF), the ratio of bytes written to the media to bytes written by the host. A high WAF degrades throughput, tail latency, and endurance [1].

A decades-long line of work observes that WAF collapses when each erase block holds data of a single *lifetime*: if everything in a block dies at about the same time, the block is reclaimed with little or no copying [2], [3]. The obstacle is information: the device does not know application lifetimes, only the host does. NVMe *Flexible Data Placement* (FDP) [4] closes this gap. The host attaches a *placement* hint to each write, and the device co-locates writes that share a hint into the same *Reclaim Unit* (RU). Crucially, placement is only a *hint*:

a write with no directive uses a default RU, so legacy software runs unchanged. This advisory, block-interface-preserving design is the key distinction from Zoned Namespaces (ZNS) [5], which replaces the block interface with append-only zones, and a standardization of multi-streamed SSDs [6].

The problem. Despite strong industry momentum, FDP is hard to study in academia: FDP-capable drives are scarce, and existing open SSD emulators do not implement it. FEMU [7] and MQSim [8] model flash timing but expose no FDP interface, and neither presents the feature to a real, unmodified NVMe stack. Researchers therefore cannot answer even basic questions (*when*, and by *how much*, does FDP help?) on commodity machines.

Our approach. We build on NVMeVirt [9], which fabricates a virtual NVMe controller at the PCIe layer inside a Linux kernel module, so the device is driven by the *unmodified* in-kernel NVMe driver and standard tools. NVMeVirt ships conventional-SSD, ZNS, and key-value targets but not FDP. We add a complete FDP device model and use it to quantify FDP’s benefit on commodity hardware with no special drivers.

Contributions.

- A functional NVMe FDP device model for NVMeVirt: Identify-level capability advertisement, the four FDP log pages (configurations, RUH usage, statistics, events) and both FDP features, host-directed placement into per-RUH reclaim units, invalid-placement handling, and FDP event reporting (§IV), validated against `nvme-cli`.
- An evaluation methodology and a two-phase synthetic lifetime workload that drive the emulated device entirely through the standard NVMe passthrough interface (§V).
- A characterization showing up to 61% WAF reduction ($2.5\times$ endurance) from placement, together with a sweep that explains how the benefit depends on over-provisioning and workload skew, and therefore *when* FDP is worthwhile; and a confirmation of the effect on a RocksDB-like LSM workload that also shows how many reclaim units a deployment needs (§VI).

II. BACKGROUND AND MOTIVATION

A. Garbage collection and write amplification

A flash translation layer (FTL) maps logical pages to physical pages and writes updates to a clean “write frontier.” When

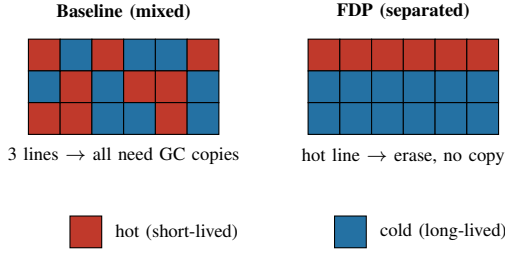


Fig. 1. Lifetime mixing. Without placement, hot rewrites strand cold survivors in many lines, forcing repeated GC copies; FDP places lifetimes in separate lines, so a hot line is reclaimed with no copying.

clean space runs low, GC selects a victim block (typically lowest valid-page count), copies its valid pages to the frontier, and erases it. If a victim mixes soon-to-die and long-lived data, the long-lived pages are copied *repeatedly* across GC cycles, inflating WAF. Over-provisioning (OP), the physical capacity beyond the exported logical capacity, gives GC more slack and lowers WAF, at the cost of usable space. The governing tension is therefore between how full the device is (set by OP) and how mixed each block is (set by lifetime separation).

B. Why placement helps: the mixing problem

Figure 1 illustrates the effect that FDP exploits. Under a skewed hot/cold workload, a placement-agnostic device interleaves hot and cold logical addresses into the same physical lines (left). When the hot data is rewritten, it leaves holes scattered across *many* lines, each still pinning a few cold survivors; GC must copy those survivors before it can erase, and it pays this cost again every time the line refills. With placement (right), hot and cold data occupy separate lines from the start; a hot line becomes *entirely* invalid and is reclaimed with *zero* copying, while the cold lines are rarely touched. The GC algorithm itself does not change. The host has simply stopped mixing lifetimes within a line.

C. NVMe Flexible Data Placement

FDP [4] adds host-directed placement to the NVMe command set, scoped to an *endurance group*. The host references a Reclaim Unit Handle (RUH) through a placement identifier carried in the write command’s directive fields; the device steers all writes sharing an RUH into the same RU, the granularity at which media is reclaimed. FDP is exposed through Identify fields, four log pages (configurations, RUH usage, statistics, events), and two features (FDP, FDP-events). A write without a placement directive uses a default RUH, so unmodified software keeps working. This property is what sets FDP apart from ZNS, and it makes FDP an attractive incremental path to lifetime-aware placement.

III. RELATED WORK

Lifetime-aware placement. Clustering data by update frequency to lower cleaning cost originates in log-structured file systems [3] and early flash cleaning policies [2]. Multi-streamed SSDs [6] first exposed a host placement hint at the

TABLE I
HOST-DIRECTED PLACEMENT INTERFACES.

Mechanism	Block iface	Placement	Legacy host
Conventional SSD	yes	none (device)	yes
Multi-stream [6]	yes	stream hint	yes
Open-Channel [13]	no	full geometry	no
ZNS [5]	no	append zones	no
FDP [4]	yes	RUH hint	yes

device interface; subsequent systems automated stream assignment from access patterns (AutoStream [10]), file-system hints (FStream [11]), or program contexts (PCStream [12]). FDP [4] generalizes streams into a standardized, endurance-group-scoped mechanism with an explicit reclaim-unit model and statistics/event reporting. Open-Channel SSDs [13] and ZNS [5] push placement control further by exposing physical geometry or append-only zones, at the cost of host complexity and a broken block interface; FDP keeps the block interface and treats placement as advisory. Lifetime separation also appears one level up the stack, for example in the key/value separation of WiscKey [14] and in the flash-friendly design of F2FS [15]; these are complementary hint *sources* for an FDP device. Table I situates these mechanisms.

SSD emulation. FEMU [7] is a QEMU-based NVMe flash emulator and MQSim [8] a detailed discrete-event device simulator; both model timing but expose no FDP interface, and a simulator is not driven by the real OS stack. NVMeVirt [9] instead emulates at the PCIe layer inside the kernel, so the unmodified NVMe driver binds to it and userspace sees an ordinary `/dev/nvme*`. We extend NVMeVirt precisely for this property: an FDP model that real tools (`nvme-cli`, `passthrough ioctl`s) exercise without modification. To our knowledge this is the first open FDP device model usable by an unmodified host stack.

IV. NVMEVIRT FDP EXTENSION

NVMeVirt presents a virtual NVMe controller at the PCIe layer: a dispatcher thread polls the controller’s doorbell registers, decodes each submission-queue entry, and routes it either to the admin handler or, for I/O, through a per-namespace handler (`ns->proc_io_cmd`) to a pluggable FTL. We add FDP as a new namespace target and reuse this structure wholesale (Fig. 2). Our FTL is forked from NVMeVirt’s conventional-SSD FTL, which already supplies everything FDP’s data path needs: a write frontier, super-block *lines* as the reclaim granularity, line-based garbage collection, and a NAND timing model.

The single substantive change to the data path is to keep *one write frontier per reclaim unit handle* (eight in our configuration) rather than one global frontier. Each handle owns its own open line, so every reclaim unit it opens starts out separate from the others. We therefore expose all handles as the *initially isolated* reclaim-unit-handle type, the FDP mode in which a newly referenced reclaim unit does not share media with any other handle, which is exactly the behavior our one-

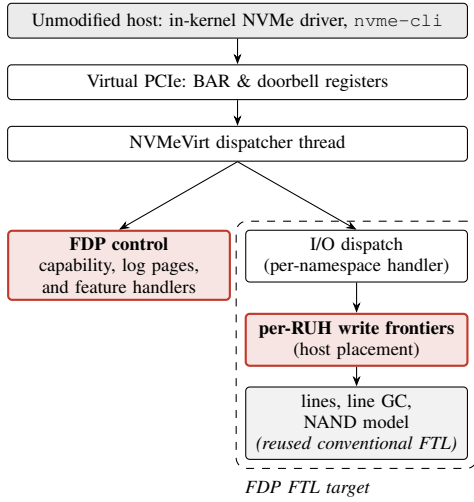


Fig. 2. The FDP target reuses NVMeVirt’s PCIe front end, dispatcher, and the conventional FTL’s lines, garbage collection, and timing model. The shaded boxes are the only additions: the FDP control surface and the per-RUH write frontiers.

line-per-handle design provides. On each write we read the command’s placement directive: a placement-tagged write is steered to the addressed RUH’s frontier, while an untagged write falls back to a default RUH. Writes that carry different placement identifiers therefore land in different physical lines, so it is the host, not the device, that decides which data shares a reclaim unit. We leave garbage collection untouched and placement-agnostic, so it relocates survivors to a shared frontier just as before. This choice is deliberate, because it ensures that any benefit we measure comes from placement alone and not from a smarter cleaning policy.

Around this data path we add the FDP control surface a standards-compliant host expects, so an unmodified Linux stack and `nvme-cli` drive the device without changes. The controller and namespace advertise FDP in Identify; the four FDP log pages report the configuration, per-RUH usage, the host/media/erase byte counters, and a ring of recent events; and the two FDP features expose the enable state and a settable event mask. Because host writes increment both the host- and media-written counters while GC copies increment only the media counter, the statistics log yields the write-amplification factor $WAF = MBMW/HBMW$ directly, and this is the quantity our evaluation reads back. We confirmed with `nvme-cli` that the capability bits, all of the log pages, and both features decode correctly, that a placement directive marks the addressed RUH as host-specified, and that an out-of-range identifier is rejected and recorded as an FDP event.

V. METHODOLOGY

We run NVMeVirt with the FDP target inside a lightweight VM that boots the host kernel, so the build is exercised by the real Linux NVMe stack and `nvme-cli`. Table II lists the configuration.

Metric. WAF is read from the FDP Statistics log. It depends only on FTL logic and the workload, not on wall-clock timing,

TABLE II
EXPERIMENTAL CONFIGURATION.

Parameter	Value
Emulator / target	NVMeVirt + FDP, SAMSUNG_FDP
Host & guest kernel	Linux 6.12 (virtme-ng)
Device capacity	64 MiB, 4 FTL partitions
Reclaim unit handles	8 (1 reclaim group, 1 endurance grp)
Intrinsic over-prov.	$\approx 7\%$
Write size	32 KiB via NVMe passthrough
Hot region / hot writes	20% of LBAs / 90% of writes (default)
Metric	WAF = MBMW/HBMW (statistics log)

so it is valid under CPU-only emulation; it also maps directly to endurance, $\text{endurance gain} = WAF_{\text{base}}/WAF_{\text{FDP}}$.

Workload. A two-phase generator models two lifetime classes. First a *random-order fill* writes the working set once; random order is essential so that, *without* placement, hot and cold addresses interleave into the same lines (Fig. 1). Then a *churn* phase rewrites the hot set repeatedly. We compare two configurations on the *same* device and workload: a **baseline** issuing no placement directive (all data to the default RUH), and **FDP** placing hot data in one RUH and cold data in another. WAF is measured as a *delta over the churn phase only* (statistics snapshotted after the fill), so the near-unit WAF of the fill does not dilute the steady-state number; each measurement uses a freshly loaded device with reset counters. The design isolates a single variable, namely whether writes carry a placement directive, so any difference we see is attributable to placement alone.

Device sizing. We deliberately use a small device so GC reaches steady state at a tractable write volume under emulation; the block geometry is scaled so that physical capacity tracks the requested logical capacity (the intended $\approx 7\%$ OP) rather than leaving so much implicit over-provisioning that GC never runs. Trends are geometry-independent; absolute WAF scales with OP and working set, which is exactly what the sweeps in §VI characterize.

VI. EVALUATION

We answer four questions: (1) how much does placement reduce write amplification at a representative operating point; (2) how does the benefit vary with over-provisioning; (3) how does it vary with workload separability; and (4) does the effect carry over to a realistic, multi-stream access pattern. Across all of them the result is one-directional: FDP *reduces* write amplification in every configuration we measured and never increases it, by 34–61% where garbage collection is active (a $1.5\text{--}2.5\times$ endurance gain). The size of the win grows with workload separability and garbage-collection pressure and shrinks toward zero when either is absent. A built-in control pins the cause to placement rather than any artifact: running FDP with a single reclaim unit (so no lifetime can be separated) reproduces the baseline WAF exactly (§VI-E).

A. Write-amplification reduction

At the operating point (90% working set, 90% of writes into the 20% hot region), both configurations issue identical

TABLE III
OPERATING POINT: IDENTICAL HOST WRITES, 47% LESS MEDIA TRAFFIC AND 49% FEWER ERASED BYTES UNDER FDP.

	Δ HBMW	Δ MBMW	MBE	WAF
Baseline	20.0 MiB	120.4 MiB	118.0 MiB	6.02
FDP	20.0 MiB	63.4 MiB	60.5 MiB	3.17

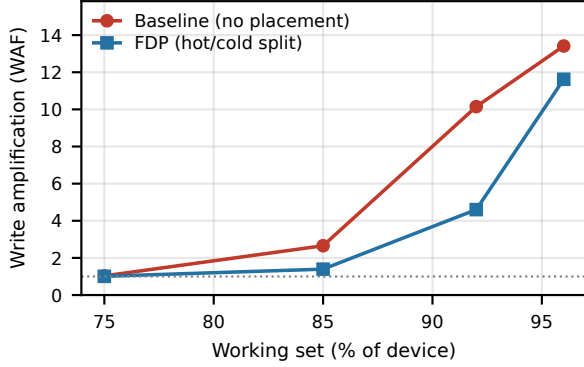


Fig. 3. WAF vs. over-provisioning (working-set size, fixed skew). FDP’s advantage peaks at moderate OP and shrinks at both extremes.

host writes, but the baseline pushes $6.0\times$ that volume to the media against $3.2\times$ for FDP, a 47% reduction (Table III). The bytes-erased counter (MBE) points to the cause: FDP erases 49% fewer bytes, which means GC runs far less often because hot lines are reclaimed without copying. The baseline pays its cost copying the cold pages that are stranded in hot-dominated lines, which is exactly the mixing that FDP avoids.

B. Effect of over-provisioning

Figure 3 sweeps the working-set size (hence free space/OP) at fixed skew. The FDP benefit is largest at *moderate* OP: with abundant free space (75% working set, $\sim 33\%$ OP) GC seldom runs and neither configuration amplifies, while under extreme pressure (96%, $\sim 4\%$ OP) the device is nearly full and even FDP must copy to make room. In between, FDP roughly halves WAF; at a 92% working set, for instance, it drops from $10.1\times$ to $4.6\times$, a 55% reduction. This non-monotonic shape matches the intuition that placement helps only when GC is both active and avoidable, and it tells a practitioner the regime in which FDP pays off.

C. Effect of workload skew

Figure 4 varies the fraction of writes hitting the hot region at fixed working set. The baseline WAF stays essentially flat at about 5.5, because mixing lifetimes hurts it regardless of skew, while FDP’s WAF falls steadily as the workload becomes more separable, from $3.95\times$ at 70% to $2.17\times$ at 95%. The reduction grows from 31% to 61%, a $1.5\times$ to $2.5\times$ endurance gain (Fig. 5). In other words, the cleaner the lifetime separation the host can express, the more FDP delivers, which is the core FDP value proposition shown directly.

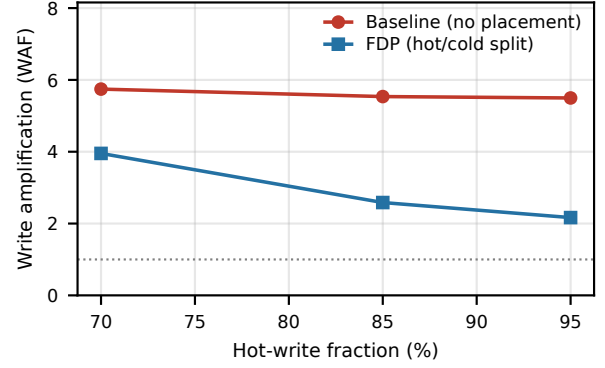


Fig. 4. WAF vs. workload skew (hot-write fraction, fixed working set). Baseline is flat; FDP improves monotonically as lifetimes separate.

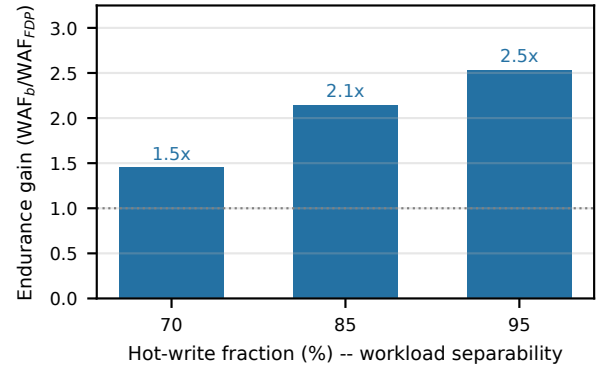


Fig. 5. Endurance gain ($WAF_{\text{base}}/WAF_{\text{FDP}}$) across the skew sweep, reaching $2.5\times$ at the most separable point.

D. Variance

Table IV repeats the operating point under two random seeds, at the lighter per-point churn used throughout the sweeps. The headline of Table III uses a deeper churn that drives the device closer to steady state, which is why its absolute WAF sits a little higher; the baseline-to-FDP ratio, which is what we report, is consistent across both. Across seeds the baseline and FDP WAF each move by only about $\pm 2\%$, well below the 2 to $3\times$ baseline-to-FDP gaps, so single-seed points are representative. The consolidated sweep data appear in Table V.

E. A RocksDB-like LSM workload

The sweeps above use a controlled two-class workload. To check whether the same effect appears on a realistic access pattern, and to use more than two reclaim units, we drive the device with an I/O stream modeled on a log-structured-merge (LSM) tree, the structure behind RocksDB and the canonical FDP use case. Under leveled compaction, level i holds on the order of T^i times as much data as the top level (fanout T), and data in a level is rewritten about once per level-fill, which happens roughly T times more often than in the level below. Data lifetime therefore tracks the level: the top level is small

TABLE IV
SEED VARIANCE AT THE OPERATING POINT ($\pm 2\%$).

Seed	Baseline WAF	FDP WAF
1	5.41	2.57
2	5.66	2.38

TABLE V
CONSOLIDATED SWEEP RESULTS. OP SWEEP AT FIXED 90% SKEW; SKEW SWEEP AT FIXED 90% WORKING SET.

Sweep	Point	Base	FDP	Reduc.	Endur.
OP	75% WS ($\sim 33\%$ OP)	1.03	1.01	2%	1.0 $\times$
OP	85% WS ($\sim 18\%$ OP)	2.66	1.40	47%	1.9 $\times$
OP	92% WS ($\sim 9\%$ OP)	10.15	4.60	55%	2.2 $\times$
OP	96% WS ($\sim 4\%$ OP)	13.41	11.62	13%	1.2 $\times$
Skew	70% hot writes	5.74	3.95	31%	1.5 $\times$
Skew	85% hot writes	5.54	2.59	53%	2.1 $\times$
Skew	95% hot writes	5.50	2.17	61%	2.5 $\times$

and churns quickly, the bottom level is large and is rarely rewritten, and adjacent levels differ in update rate by about T . The *level* is thus a natural lifetime class, and “one level per reclaim unit” is exactly the mapping that RocksDB-on-FDP deployments use. We generate this stream directly from the level geometry (default $T = 4$, four levels) rather than running RocksDB, so the experiment stays self-contained while its hot/cold structure comes from the data structure rather than an arbitrary knob.

Figure 6 sweeps how many reclaim unit handles hold the levels, from one (all data in a single reclaim unit, equivalent to the baseline) up to one per level. With a single handle, FDP reproduces the baseline WAF of 6.6 exactly—a built-in sanity check, since one reclaim unit cannot separate anything. WAF then falls monotonically as more levels get their own handle, reaching 4.3 with one handle per level: a 34% reduction, or a 1.5 $\times$ endurance gain. The curve has a clear knee—isolating the fast-churning top level first yields the largest single improvement and each additional handle helps less—which is direct guidance on how many reclaim units a deployment actually needs. The reduction is smaller than the two-class operating point (47%) because leveled compaction spreads churn across all levels rather than concentrating it in a tiny hot region; even so, it confirms the placement effect on a workload whose lifetimes arise from a real data structure rather than a synthetic hot/cold knob.

F. Discussion and limitations

The results isolate the placement effect: baseline and FDP differ only in whether writes carry a directive, on the same device and FTL. Two limitations follow from the setup. First, we report WAF and endurance rather than throughput or latency, because NVMeVirt’s timing model is not faithful under pure CPU emulation, so performance numbers would require a bare-metal run. This is a natural next step now that the functional model exists. Second, our workloads are generated rather than captured: the two-class sweeps use the standard

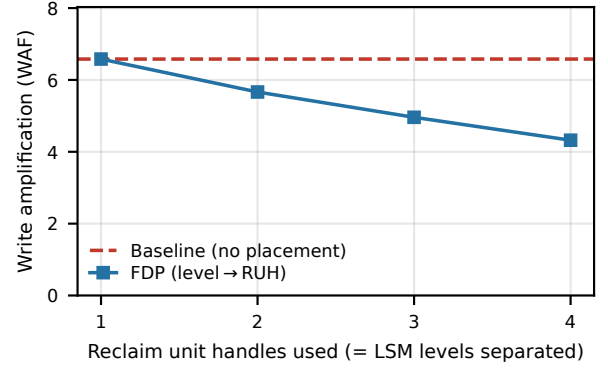


Fig. 6. RocksDB-like LSM workload: WAF vs. the number of reclaim unit handles used to separate LSM levels. One handle reproduces the baseline; the first split gives the largest single gain, and one handle per level cuts WAF by 34%.

TABLE VI
ROCKSDB-LIKE LSM WORKLOAD ($T=4$, 4 LEVELS): WAF VS. RECLAIM UNIT HANDLES USED. $N=1$ (SINGLE RECLAIM UNIT) REPRODUCES THE BASELINE.

RUHs (N)	Baseline	FDP	Reduc.	Endur.
1	6.58	6.58	0%	1.0 $\times$
2	6.58	5.66	14%	1.2 $\times$
3	6.58	4.96	25%	1.3 $\times$
4	6.58	4.32	34%	1.5 $\times$

hot/cold lifetime abstraction, and the LSM experiment is driven by a model of leveled compaction rather than a live RocksDB instance. Replaying a captured trace from a running application would further strengthen external validity, and our standard-interface design makes that straightforward. Finally, GC currently reclaims to a shared frontier, so a placement-preserving GC is one avenue to push WAF lower still.

VII. CONCLUSION

We added a functional NVMe FDP device model to NVMeVirt and used it to quantify host-directed, lifetime-aware data placement. On a skewed hot/cold workload, FDP cuts write amplification by up to 61% (2.5 $\times$ endurance) purely by keeping lifetimes out of each other’s reclaim units, with a benefit that depends predictably on over-provisioning and workload skew. The emulator makes FDP experimentation possible without FDP hardware and on an unmodified host stack, which lowers the barrier to placement-aware storage research. The extension and all of its artifacts, including the workload, the harness, the raw data, and the figures, are publicly available in our NVMeVirt fork.

REFERENCES

- [1] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, 1st ed. Arpaci-Dusseau Books, 2018.
- [2] M.-L. Chiang, P. C. H. Lee, and R.-C. Chang, “Using data clustering to improve cleaning performance for flash memory,” *Software: Practice and Experience*, vol. 29, no. 3, pp. 267–290, 1999.

- [3] M. Rosenblum and J. K. Ousterhout, "The design and implementation of a log-structured file system," *ACM Transactions on Computer Systems*, vol. 10, no. 1, pp. 26–52, 1992.
- [4] NVM Express, "NVM Express base specification, technical proposal 4146a: Flexible data placement," <https://nvmexpress.org/>, 2022.
- [5] M. Bjørling, A. Aghayev, H. Holmberg, A. Ramesh, D. L. Moal, G. R. Ganger, and G. Amvrosiadis, "ZNS: Avoiding the block interface tax for flash-based SSDs," in *Proc. 2021 USENIX Annual Technical Conf. (ATC)*, 2021, pp. 689–703.
- [6] J.-U. Kang, J. Hyun, H. Maeng, and S. Cho, "The multi-streamed solid-state drive," in *Proc. 6th USENIX Workshop Hot Topics in Storage and File Systems (HotStorage)*, 2014.
- [7] H. Li, M. Hao, M. H. Tong, S. Sundararaman, M. Bjørling, and H. S. Gunawi, "The CASE of FEMU: Cheap, accurate, scalable and extensible flash emulator," in *Proc. 16th USENIX Conf. File and Storage Technologies (FAST)*, 2018, pp. 83–90.
- [8] A. Tavakkol, J. Gómez-Luna, M. Sadrosadati, S. Ghose, and O. Mutlu, "MQSim: A framework for enabling realistic studies of modern multi-queue SSD devices," in *Proc. 16th USENIX Conf. File and Storage Technologies (FAST)*, 2018, pp. 49–66.
- [9] S.-H. Kim, J. Shim, E. Lee, S. Jeong, I. Kang, and J.-S. Kim, "NVMeVirt: A versatile software-defined virtual NVMe device," in *Proc. 21st USENIX Conf. File and Storage Technologies (FAST)*, 2023, pp. 379–394.
- [10] J. Yang, R. Pandurangan, C. Choi, and V. Balakrishnan, "AutoStream: Automatic stream management for multi-streamed SSDs," in *Proc. 10th ACM Int. Systems and Storage Conf. (SYSTOR)*, 2017.
- [11] E. Rho, K. Joshi, S.-U. Shin, N. J. Shetty, J.-Y. Hwang, S. Cho, D. D. G. Lee, and J. Jeong, "FStream: Managing flash streams in the file system," in *Proc. 16th USENIX Conf. File and Storage Technologies (FAST)*, 2018, pp. 257–264.
- [12] T. Kim, D. Hong, S. S. Hahn, M. Chun, S. Lee, J. Hwang, J. Lee, and J. Kim, "Fully automatic stream management for multi-streamed SSDs using program contexts," in *Proc. 17th USENIX Conf. File and Storage Technologies (FAST)*, 2019, pp. 295–308.
- [13] M. Bjørling, J. González, and P. Bonnet, "LightNVM: The Linux open-channel SSD subsystem," in *Proc. 15th USENIX Conf. File and Storage Technologies (FAST)*, 2017, pp. 359–374.
- [14] L. Lu, T. S. Pillai, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "WiscKey: Separating keys from values in SSD-conscious storage," in *Proc. 14th USENIX Conf. File and Storage Technologies (FAST)*, 2016, pp. 133–148.
- [15] C. Lee, D. Sim, J.-Y. Hwang, and S. Cho, "F2FS: A new file system for flash storage," in *Proc. 13th USENIX Conf. File and Storage Technologies (FAST)*, 2015, pp. 273–286.